

Project Report: Infrastructure and Virtualization

Deployment and Resilience Analysis of a Kubernetes-Based Nextcloud Cluster

Muhammad Omais Rana

Master in Internet of Things (IoT)

University of Marie and Louis Pasteur, UFR STGI

April 3, 2026

Contents

1	Introduction	2
2	System Architecture	2
2.1	Networking and CNI	2
2.2	Persistence Layer	2
3	Deployment and Implementation	2
4	Resilience Analysis: Disaster Recovery Case Study	2
4.1	Incident Summary	3
4.2	Troubleshooting and Resolution	3
4.3	Results	3
5	Conclusion	3
6	Appendix: File Manifest Checklist	3

1 Introduction

This project explores the deployment of a resilient, multi-tier private cloud storage solution using Kubernetes. By integrating Nextcloud with a PostgreSQL database and an NFS-backed persistence layer, the architecture ensures that data remains accessible and intact even across ungraceful node shutdowns or hardware failures.

2 System Architecture

The cluster is built on a distributed model using three Virtual Machines. The separation of concerns between the control plane and data plane ensures stability.

Table 1: Cluster Node Configuration

Node Name	Role	IP Address	Primary Services
control	Master	192.168.80.100	API Server, ETCD, NFS Server
wrk1	Worker	192.168.80.101	Nextcloud, PostgreSQL
wrk2	Worker	192.168.80.102	Nginx Reverse Proxy

2.1 Networking and CNI

Connectivity is managed by the Calico CNI (Container Network Interface). It utilizes BGP peering for internal routing and IPIP encapsulation (Protocol 4) to tunnel pod traffic across the host network.

2.2 Persistence Layer

Storage persistence is achieved through a Network File System (NFS) hosted on the control node. Persistent Volume Claims (PVC) map application data directories to dedicated sub-paths on the NFS share, ensuring that container restarts do not result in data loss.

3 Deployment and Implementation

The deployment utilizes declarative YAML manifests. The sequence of operation involves initializing the storage backend, followed by the database, and finally the application layer.

```
1 # Apply storage and database manifests
2 kubectl apply -f postgres-complex.yaml
3
4 # Apply application and ingress manifests
5 kubectl apply -f nextcloud-complex.yaml
6 kubectl apply -f nginx-complex.yaml
```

Listing 1: Cluster Initialization Commands

4 Resilience Analysis: Disaster Recovery Case Study

A critical component of this lab involved testing system behavior during a total power failure.

4.1 Incident Summary

The host machine experienced a sudden shutdown, causing all virtual nodes to lose power simultaneously. Upon restoration, the cluster entered a recovery state where pods were rescheduled automatically but remained initially unreachable.

4.2 Troubleshooting and Resolution

Several technical hurdles were identified and resolved:

- **Stale NFS Handles:** The PostgreSQL instance was prevented from starting by a stale `postmaster.pid` lock.
- **CNI Instability:** The IPIP tunnel was blocked by `iptables` rules that reset to default policies upon reboot.
- **Application Security:** Nextcloud's `trusted_domains` security required manual updates to recognize new post-reboot IP signatures.

4.3 Results

Following manual reconciliation of the network fabric and storage locks, the application successfully recovered. Full data persistence was verified through the presence of user files uploaded prior to the failure.

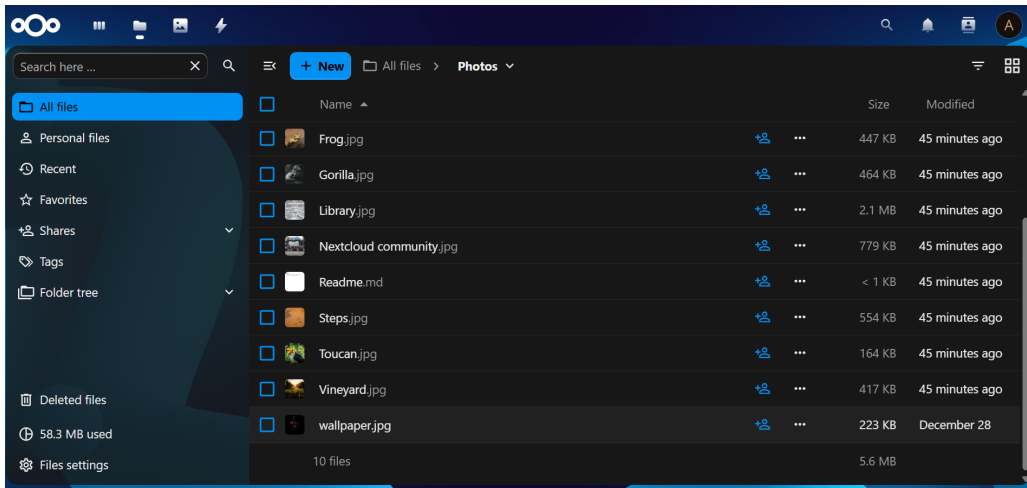


Figure 1: Nextcloud Dashboard showing data persistence and active file storage after system recovery.

5 Conclusion

This deployment successfully demonstrates the feasibility of high-availability IoT cloud services. The decoupling of the storage layer via NFS proved essential for data survival, while the incident highlighted the importance of network fabric monitoring in distributed systems.

6 Appendix: File Manifest Checklist

The following files are included in the project archive:

1. `postgres-complex.yaml`: PV, PVC, Service, and Deployment for PostgreSQL.

2. `nextcloud-complex.yaml`: Deployment and NodePort Service for the web app.
3. `nginx-complex.yaml`: Reverse proxy configuration.
4. `config.php`: Finalized application configuration showing trusted domains.